Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «СПЕЦИАЛЬНОЕ МАШИНОСТРОЕНИЕ»

КАФЕДРА «РАКЕТНЫЕ И ИМПУЛЬСНЫЕ СИСТЕМЫ» (СМ-6)

Лабораторная работа №2

ПО ДИСЦИПЛИНЕ:

«Компьютерное моделирование»

НА ТЕМУ:

«Задача *N* тел: прямой метод и метод Барнса-Хата»

Выполнили: студенты группы СМ6-99 _____ (подпись, дата) Е.В. Швецова (И.О. Фамилия)
_____ (подпись, дата) А.А. Довыденко (И.О. Фамилия)

Проверил _____ (подпись, дата) В. А. Федулов (И.О. Фамилия)

Москва, 2026 г.

Содержание

1 Введение 3
2 Математическая модель 3
3 Результаты 4
3.1 Тест интегратора: задача двух тел 4
3.2 Вращающийся диск 5
3.3 Время работы двух методов 6
3.4 Анализ точности по методу Барнса-Хата 7
3.5 Выводы 8
4 Заключение 9
Список литературы 10
Приложение 11

1 Введение

Задача N тел — одна из фундаментальных задач математической физики. Она описывает движение системы материальных точек, взаимодействующих по закону всемирного тяготения. Задача возникает в астрофизике (движение планет, звёзд, галактик), космологии (эволюция крупномасштабной структуры Вселенной), молекулярной динамике (взаимодействие атомов и молекул) и физике плазмы.

Аналитическое решение задачи N тел существует только для $N = 2$ — классическая задача двух тел, решённая ещё Ньютоном. Для $N \geq 3$ аналитического решения в общем случае нет, поэтому применяются численные методы. Прямой метод расчёта сил требует $O(N^2)$ операций: для каждой из N частиц вычисляется взаимодействие со всеми остальными. При $N = 10^6$ это порядка 10^{12} операций, что делает прямой расчёт вычислительно невозможным за разумное время.

Алгоритм Барнса-Хата — приближённый метод, позволяющий снизить сложность до $O(N \log N)$. Идея: если группа частиц находится далеко от рассматриваемой точки, всю группу можно заменить одной эффективной частицей с суммарной массой, расположенной в центре масс группы. Это позволяет при $N = 10^6$ сократить число операций до $\sim 10^7$ — в 100000 раз быстрее.

Цель работы — реализовать численную модель гравитационного взаимодействия N тел в двумерном пространстве и сравнить два метода расчёта сил: прямой метод $O(N^2)$ и алгоритм Барнса-Хата $O(N \log N)$ по времени работы и точности. Для реализации использован язык Python.

2 Математическая модель

Движение каждой частицы описывается вторым законом Ньютона:

$$m_i \cdot a_i = F_i,$$

где m_i — масса i-й частицы, a_i — её ускорение, F_i — суммарная сила, действующая на частицу.

Сила гравитационного взаимодействия между частицами i и j определяется законом всемирного тяготения:

$$F_{ij} = -G \cdot \left(\frac{m_i \cdot m_j}{|r_i - r_j|^3}\right) \cdot (r_i - r_j),$$

где G — гравитационная постоянная, r_i и r_j — радиус-векторы частиц.

Полная сила на i-ю частицу равна сумме вкладов от всех остальных:

$$F_i = \Sigma F_{ij}.$$

Чтобы избежать численную неустойчивость при сближении частиц вводится параметр сглаживания (softening) ε:

$$|r_i - r_j|^2 \longrightarrow |r_i - r_j|^2 + \varepsilon^2.$$

Для численного интегрирования уравнений движения используется метод velocity Verlet (интегратор Верле). Алгоритм одного шага:

1. $v_i(t + \frac{\Delta t}{2}) = v_i(t) + \frac{1}{2} \cdot a_i(t) \cdot \Delta t$ — половина обновления скорости;
2. $r_i(t + \Delta t) = r_i(t) + v_i(t + \frac{\Delta t}{2}) \cdot \Delta t$ — обновление позиции;
3. $a_i(t + \Delta t) = \frac{F_i(t + \Delta t)}{m_i}$ — пересчёт сил по новым позициям;
4. $v_i(t + \Delta t) = v_i(t + \frac{\Delta t}{2}) + \frac{1}{2} \cdot a_i(t + \Delta t) \cdot \Delta t$ — вторая половина скорости.

Метод Верле имеет второй порядок точности по времени и сохраняет энергию системы на длительных интервалах.

В алгоритме Барнса-Хата далёкую группу частиц можно заменить одной эффективной частицей с суммарной массой в её центре масс. Всё пространство моделирования заключается в квадрат. Если в квадрате находится более одной частицы, он делится на 4 подквадрата. Процесс повторяется рекурсивно до тех пор, пока в каждой ячейке не останется не более одной частицы. Каждый узел дерева хранит границы ячейки, суммарную массу и координаты центра масс.

Критерий приближения: для узла дерева размером s, находящегося на расстоянии d от рассматриваемой частицы, проверяется условие:

$$\frac{s}{d} < \theta,$$

где θ — параметр точности .

Если условие выполнено — узел приближается одной частицей. Если нет — происходит рекурсивный обход дочерних узлов.

Центр масс узла вычисляется по формуле:

$$r_{cm} = \frac{\Sigma\, m_i \cdot r_i}{d\Sigma\, m}).$$

3 Результаты

3.1 Тест интегратора: задача двух тел

Для проверки корректности интегратора Верле проведена симуляция задачи двух тел: две частицы массой 1 на расстоянии 1. Обе частицы движутся по круговым орбитам

радиусом 0,5 вокруг общего центра масс. Орбиты замкнуты, энергия сохраняется на протяжении всей симуляции.

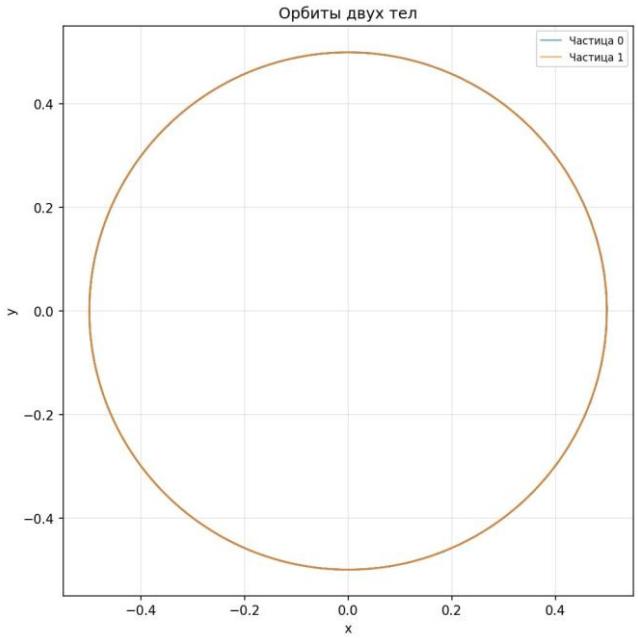


Рисунок 1. Орбиты двух тел

3.2 Вращающийся диск

Для проверки работы модели с N тел проведена симуляция вращающегося диска: 100 частиц в круге радиусом 1, общая масса 10, тангенциальные скорости (круговая скорость на каждом радиусе). Основная масса частиц удерживается в центре гравитационным притяжением, часть частиц уходит на периферию. Результаты прямого метода и Барнса-Хата визуально совпадают.

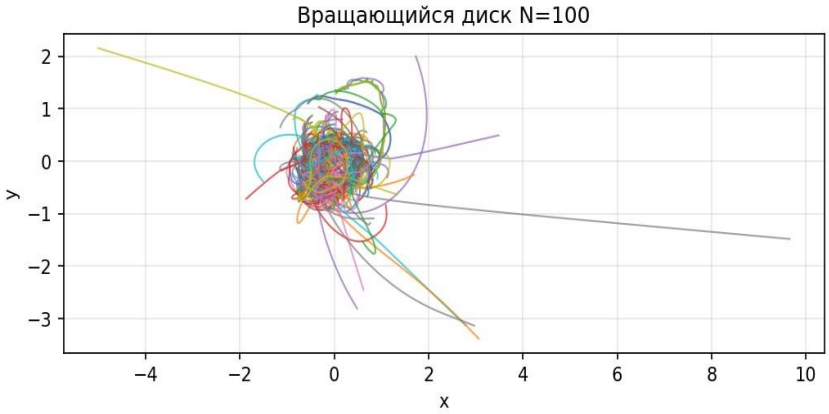


Рисунок 2. Вращающийся диск: прямой метод

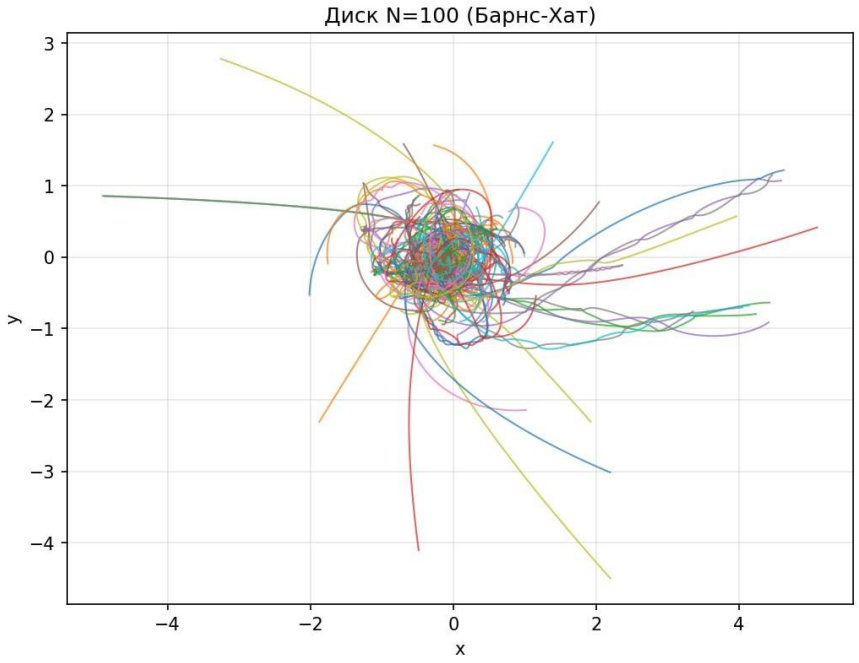


Рисунок 3. Вращающийся диск: метод Барнса-Хата

3.3 Время работы двух методов

Для исследования ускорения был проведён эксперимент: замерялось время работы обоих методов для N = 100, 300, 500, 1000, 2000. Каждый замер повторялся 3 раза, выбиралось минимальное время.

Таблица 1. Сравнение времени работы прямого метода и метода Барнса-Хата

N	Прямой (с)	Барнс-Хат (с)
100	0,0026465	0,0035696
300	0,0242916	0,0167069
500	0,0677789	0,0328740
1000	0,2728203	0,0778951
2000	1,0939457	0,18340530

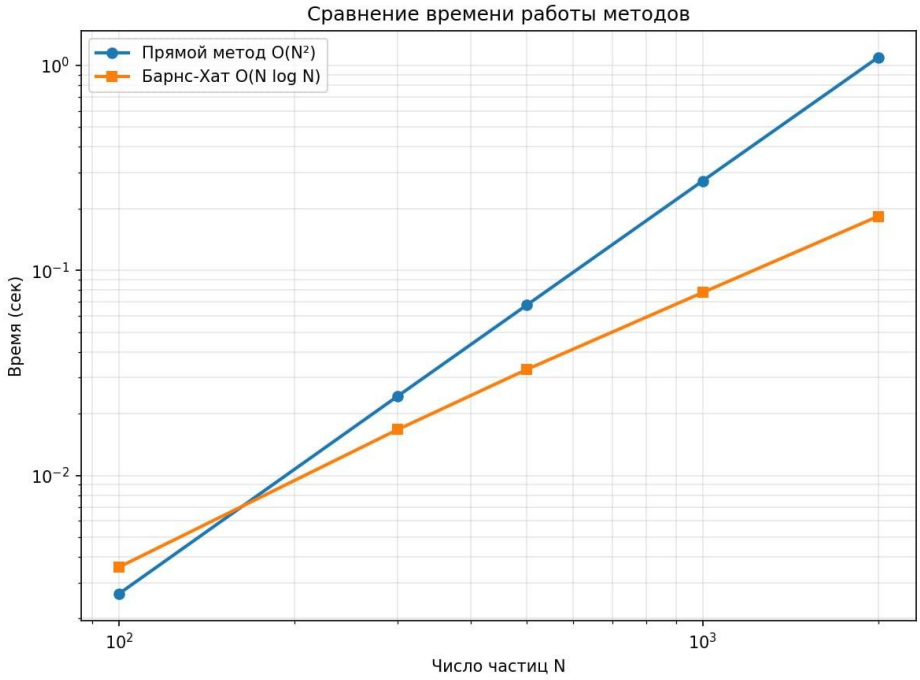


Рисунок 4. Время работы прямого метода и метода Барнса-Хата

Из таблицы 1 и рисунка 4 видно, что при $N = 100$ прямой метод быстрее, что объясняется накладными расходами на построение дерева. Однако уже при $N = 300$ Барнс-Хат быстрее прямого метода.

3.4 Анализ точности по методу Барнса-Хата

Для исследования влияния параметра θ на точность проведён эксперимент: при $N = 1000$ сравнивались результаты Барнса-Хата с прямым методом (эталон).

Таблица 2. Зависимость точности от параметра θ

θ	Средняя ошибка	Относительная ошибка
0,1	0,00026081	0,0120
0,3	0,00548715	0,2526
0,5	0,02070870	0,9533
0,7	0,05020913	2,3113
1,0	0,15894126	7,3167

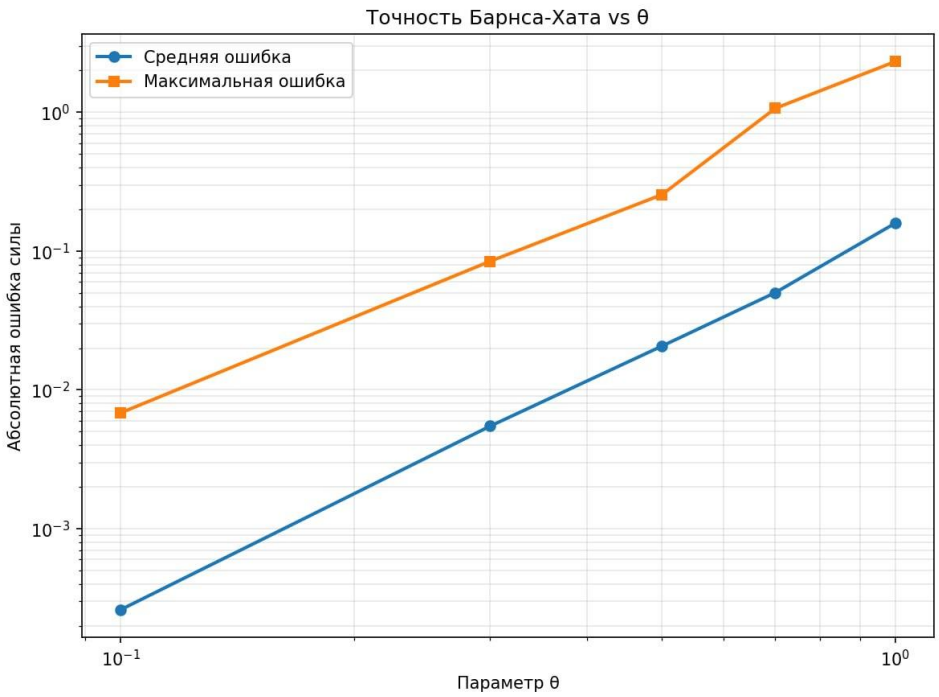


Рисунок 5. Зависимость точности от параметра θ

Из таблицы 2 и рисунка 5 видно, что при θ = 0,1 относительная ошибка минимальна, но при этом метод работает медленнее (много узлов обходится детально). При θ = 1,0 ошибка достигает 7,32%, но расчёт быстрее. Оптимальным является θ = 0,5 — ошибка меньше 1%.

3.5 Выводы

1. Реализованы прямой метод расчёта сил и интегратор Верле. Тест на двух телах подтвердил корректность: орбиты замкнуты, энергия сохраняется;
2. Реализован алгоритм Барнса-Хата на основе квадродерева. Визуальное сравнение с прямым методом показало совпадение результатов;
3. Экспериментально подтверждено ускорение вычислений: при $N > 300$ Барнс-Хат быстрее прямого метода. Ускорение растёт с ростом N;
4. Исследована зависимость точности от θ. Оптимальным является θ = 0,5 — относительная ошибка 0,95% (< 1%);
5. Полученные результаты качественно и количественно согласуются с теоретическими оценками и литературными данными.

4 Заключение

В ходе лабораторной работы была построена и исследована численная модель гравитационного взаимодействия *N* тел в двумерном пространстве. Модель реализована на Python.

В результате лабораторной работы:

- Формализована математическая модель: уравнения движения, закон тяготения, softening, интегратор Верле;
- Реализован алгоритм Барнса-Хата на основе квадродерева с критерием приближения $\frac{s}{d} < \theta$;
- Экспериментально подтверждена эффективность Барнса-Хата по сравнению с прямым методом;
- Исследована точность: при $\theta = 0{,}5$ относительная ошибка < 1%, что является оптимальным вариантом;
- Визуализация орбит и диска подтвердила корректность численной модели.

Алгоритм Барнса-Хата продемонстрировал значительное ускорение вычислений при сохранении приемлемой точности, что делает его эффективным инструментом для моделирования систем с большим числом частиц.

Список литературы

1. Федулов В.А. Лекция 6. Методы динамики частиц.
2. Barnes J., Hut P. A hierarchical O(N log N) force-calculation algorithm // Nature. — 1986. — Vol. 324. — P. 446–449.
3. NumPy Documentation [Электронный ресурс]. — URL: https://numpy.org/doc/.
4. Matplotlib Documentation [Электронный ресурс]. — URL: https://matplotlib.org/ .
5. Python Documentation [Электронный ресурс]. — URL: https://docs.python.org/3/ .

Приложение

```
import math
import random

import matplotlib.pyplot as plt

from particle import Particle
from barnes_hut import compute_forces_barnes_hut


def two_body_system(
    m1: float, m2: float, separation: float, G: float = 1.0
) -> list[Particle]:
    """Два тела на круговой орбите вокруг общего центра масс."""
    total = m1 + m2
    r1 = separation * m2 / total
    r2 = separation * m1 / total

    v1 = math.sqrt(G * m2 * m2 / (total * separation))
    v2 = math.sqrt(G * m1 * m1 / (total * separation))

    p1 = Particle(m1, -r1, 0.0, 0.0, v1)
    p2 = Particle(m2, r2, 0.0, 0.0, -v2)
    return [p1, p2]


def uniform_cluster(
    n: int, radius: float, total_mass: float, seed: int | None =
None
) -> list[Particle]:
    """Однородное скопление из n частиц в круге радиуса
radius."""
    if seed is not None:
        random.seed(seed)

    particles = []
    mass_per_particle = total_mass / n

    for _ in range(n):
        # rejection sampling: генерируем точку в квадрате,
        # пока она не попадёт в круг
        while True:
            x = random.uniform(-radius, radius)
            y = random.uniform(-radius, radius)
            if x * x + y * y <= radius * radius:
                break

        particles.append(Particle(mass_per_particle, x, y))

    return particles
```

```
def rotating_disk(
    n: int, radius: float, total_mass: float, seed: int | None =
None
) -> list[Particle]:
    """Вращающийся диск: частицы с тангенциальными
скоростями."""
    if seed is not None:
        random.seed(seed)

    particles = []
    mass_per_particle = total_mass / n
    G = 1.0

    for _ in range(n):
        # Случайная точка в круге
        while True:
            x = random.uniform(-radius, radius)
            y = random.uniform(-radius, radius)
            if x * x + y * y <= radius * radius:
                break

        r = math.sqrt(x * x + y * y)
        if r == 0:
            vx, vy = 0.0, 0.0
        else:
            # Круговая скорость внутри диска: v = sqrt(G * M(r)
/ r)
            # M(r) — масса внутри радиуса r
            mass_inside = total_mass * (r / radius) ** 2
            v = 0.5 * math.sqrt(G * mass_inside / r)
            # Тангенциальное направление: перпендикулярно (x, y)
            vx = -v * y / r
            vy = v * x / r

        particles.append(Particle(mass_per_particle, x, y, vx,
vy))

    return particles


def compute_forces_direct(
    particles: list[Particle], G: float = 1.0, softening: float
= 0.01
) -> None:
    """Считает ускорения прямым методом O(N²)."""
    n = len(particles)
    for p in particles:
        p.ax = 0.0
        p.ay = 0.0

    eps2 = softening * softening

    for i in range(n):
```

```
        pi = particles[i]
        for j in range(i + 1, n):
            pj = particles[j]
            dx = pj.x - pi.x
            dy = pj.y - pi.y
            r2 = dx * dx + dy * dy + eps2
            inv_r3 = 1.0 / (r2 * math.sqrt(r2))

            pi.ax += G * pj.mass * dx * inv_r3
            pi.ay += G * pj.mass * dy * inv_r3
            pj.ax -= G * pi.mass * dx * inv_r3
            pj.ay -= G * pi.mass * dy * inv_r3


def velocity_verlet_step(
    particles: list[Particle],
    dt: float,
    G: float = 1.0,
    method: str = "direct",
    theta: float = 0.5,
) -> None:
    """Один шаг интегратора velocity Verlet.

    Args:
        method: "direct" — прямой O(N²), "barnes_hut" — дерево
O(N log N).
        theta: параметр точности для Барнса-Хата.
    """
    # Шаг 1: половина обновления скорости
    for p in particles:
        p.vx += 0.5 * p.ax * dt
        p.vy += 0.5 * p.ay * dt

    # Шаг 2: обновление позиции
    for p in particles:
        p.x += p.vx * dt
        p.y += p.vy * dt

    # Шаг 3: пересчёт сил (выбор метода)
    if method == "direct":
        compute_forces_direct(particles, G)
    elif method == "barnes_hut":
        compute_forces_barnes_hut(particles, theta=theta)
    else:
        raise ValueError(f"Неизвестный метод: {method}")

    # Шаг 4: вторая половина скорости
    for p in particles:
        p.vx += 0.5 * p.ax * dt
        p.vy += 0.5 * p.ay * dt


def simulate(
```

```
    particles: list[Particle],
    dt: float,
    steps: int,
    G: float = 1.0,
    record_every: int = 10,
    method: str = "direct",
    theta: float = 0.5,
) -> list[list[Particle]]:
    """Полная симуляция. Возвращает историю состояний.

    Args:
        method: "direct" или "barnes_hut".
        theta: параметр точности для Барнса-Хата.
    """
    history: list[list[Particle]] = []

    # Начальный расчёт сил
    if method == "direct":
        compute_forces_direct(particles, G)
    elif method == "barnes_hut":
        compute_forces_barnes_hut(particles, theta=theta)
    else:
        raise ValueError(f"Неизвестный метод: {method}")

    for step in range(steps):
        velocity_verlet_step(particles, dt, G, method, theta)

        if step % record_every == 0:
            snapshot = [Particle(p.mass, p.x, p.y, p.vx, p.vy)
for p in particles]
            history.append(snapshot)

    return history


def plot_orbits(
    history: list[list[Particle]],
    title: str = "Орбиты N тел",
    save_path: str | None = None,
    show_legend: bool = True,
) -> None:
    """Рисует траектории всех частиц.

    Args:
        history: история состояний.
        title: заголовок.
        save_path: путь для сохранения.
        show_legend: показывать ли легенду (для N > 20 — обычно
нет).
    """
    n = len(history[0])
    plt.figure(figsize=(8, 8))
```

```
    for i in range(n):
        xs = [state[i].x for state in history]
        ys = [state[i].y for state in history]
        label = f"Частица {i}" if show_legend else None
        plt.plot(xs, ys, lw=1.0, label=label, alpha=0.7)

    plt.gca().set_aspect("equal")
    plt.grid(True, alpha=0.3)
    plt.title(title)
    if show_legend and n <= 20:
        plt.legend(fontsize=8, loc="upper right")
    plt.xlabel("x")
    plt.ylabel("y")

    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches="tight")
        print(f"График сохранён в {save_path}")

    plt.show()
    plt.close()


if __name__ == "__main__":
    # Тест 1: два тела (прямой метод)
    print("=== Тест 1: два тела ===")
    ps = two_body_system(1.0, 1.0, 1.0)
    history = simulate(ps, dt=0.01, steps=2000, record_every=10,
method="direct")
    plot_orbits(
        history,
        title="Орбиты двух тел",
        save_path="results/two_body.png",
    )

    # Тест 2: вращающийся диск (прямой метод)
    print("\n=== Тест 2: вращающийся диск N=100 ===")
    disk = rotating_disk(n=100, radius=1.0, total_mass=10.0,
seed=42)
    history_disk = simulate(
        disk, dt=0.0001, steps=20000, record_every=200,
method="direct"
    )
    plot_orbits(
        history_disk,
        title="Вращающийся диск N=100",
        save_path="results/disk.png",
        show_legend=False,
    )

    # Тест 3: тот же диск, но через Барнс-Хат
    print("\n=== Тест 3: диск N=100 (Барнс-Хат, θ=0.5) ===")
    disk_bh = rotating_disk(n=100, radius=1.0, total_mass=10.0,
seed=42)
```

```
history_bh = simulate(
    disk_bh,
    dt=0.0001,
    steps=20000,
    record_every=200,
    method="barnes_hut",
    theta=0.5,
)
plot_orbits(
    history_bh,
    title="Диск N=100 (Барнс-Хат)",
    save_path="results/disk_bh.png",
    show_legend=False,
)
```